Software Requirements Specification

Jackfruit Phase-1 — Student Record System

Tanmay T A, PES1UG24CS492 Uttam B S, PES1UG24CS508
Vibha Venkatesh Vasisht, PES1UG24CS526

Generated 2026-10-10

Contents

1 Revision History **2**

2 Introduction **3**
2.1 Purpose . . . 3
2.2 Intended Audience and Reading Suggestions . . . 3
2.3 Product Scope . . . 3
2.4 References . . . 3

3 Overall Description **3**
3.1 Product Perspective . . . 3
3.2 Product Functions . . . 3
3.3 User Classes and Characteristics . . . 3
3.4 User Classes and Characteristics . . . 3
3.4.1 Admin . . . 4
3.4.2 Teacher . . . 4
3.4.3 Student . . . 4
3.5 Operating Environment . . . 5
3.6 Design and Implementation Constraints . . . 5
3.7 Assumptions and Dependencies . . . 5

4 External Interface Requirements **5**
4.1 User Interfaces . . . 5
4.2 Software Interfaces . . . 7
4.3 Communications Interfaces . . . 8

5 Analysis Models **9**

6 System Features **9**
6.1 Onboarding . . . 9
6.1.1 Description and Priority . . . 9
6.1.2 Stimulus/Response Sequences . . . 9
6.1.3 Functional Requirements . . . 10
6.2 Edit Records . . . 11

6.2.1 Description and Priority . . . 11
6.2.2 Stimulus/Response Sequences . . . 11
6.2.3 Functional Requirements . . . 12
6.3 Mark Attendance (with constraints) . . . 12
6.3.1 Description and Priority . . . 12
6.3.2 Stimulus/Response Sequences . . . 12
6.3.3 Functional Requirements . . . 12
6.4 Mark Grades (with rubric) . . . 12
6.4.1 Description and Priority . . . 12
6.4.2 Stimulus/Response Sequences . . . 12
6.4.3 Functional Requirements . . . 12
6.5 Retrieval . . . 12
6.5.1 Description and Priority . . . 12
6.5.2 Stimulus/Response Sequences . . . 12
6.5.3 Functional Requirements . . . 12
6.6 Role-Based Access . . . 13
6.6.1 Description and Priority . . . 13
6.6.2 Stimulus/Response Sequences . . . 13
6.6.3 Functional Requirements . . . 14

7 Other Nonfunctional Requirements 16
7.1 Performance Requirements . . . 16
7.2 Safety Requirements . . . 16
7.3 Security Requirements . . . 16
7.4 Software Quality Attributes . . . 16
7.5 Security Requirements . . . 16
7.6 Business Rules . . . 18

8 Other Requirements 18

9 Appendix A: Glossary 18

10 Appendix B: Field Layouts 18
10.1 Student . . . 18

11 Appendix C: Requirement Traceability Matrix 18

12 Appendix D: Stakeholder Discussion Log 18
12.1 Persona . . . 19
12.2 Discussion 1: Discovery Call (Monday, 14 September 2026) . . . 19
12.3 Discussion 2: Deployment and Constraints (Thursday, 24 September 2026) . . . 20
12.4 Discussion 3: Sign-off (Tuesday, 6 October 2026) . . . 21

1 Revision History

Name	Date	Reason For Changes	Version
		Initial draft	0.1

Name	Date	Reason For Changes	Version

Add a row every time a substantive change is made (not every typo fix). Keep this current — it's a required section.

2 Introduction

2.1 Purpose

TODO

2.2 Intended Audience and Reading Suggestions

TODO

2.3 Product Scope

TODO

2.4 References

TODO

3 Overall Description

3.1 Product Perspective

TODO

3.2 Product Functions

- Onboarding (student/staff registration)
- Edit records
- Mark attendance (with constraints)
- Mark grades (with rubric)
- Retrieval / search
- Role-based access control

3.3 User Classes and Characteristics

3.4 User Classes and Characteristics

The Student Record System has three user classes: Admin, Teacher and Student. Every account holds exactly one of these roles, and access to functions and records depends on that role (Section 5.6). Accounts are created only by an Admin (Section 5.1), so a person cannot register themselves into any user class. The detailed permissions of each class are specified in Section 5.6 and are not repeated here.

The descriptions of technical proficiency and frequency of use below are assumptions made for this project, not established requirements. The user classes, their access levels and their access limitations follow from Sections 5.1 and 5.6.

3.4.1 Admin

- **Purpose:** Administers the system. The Admin creates and maintains user accounts and keeps the student data accurate.
- **Main functions:** Manages Student and Teacher accounts (create, update, deactivate), assigns and changes roles, and manages student records, attendance and grades.
- **Access level:** Highest privilege. The Admin can access all relevant student and teacher records.
- **Information accessed:** Account details of all users, and the records, attendance and grades of all students.
- **Technical proficiency (assumed):** Moderate. The Admin is expected to be comfortable with administrative web applications and data entry, but needs no programming knowledge.
- **Frequency of use (assumed):** Periodic. Use is concentrated around account setup and record maintenance, with ad hoc use whenever accounts or records need correction.
- **Important characteristics and limitations:** Only the Admin can manage accounts and roles. One pre-seeded Admin account exists from initial system setup. The Admin never sets or knows a user's password, because each user sets their own.

3.4.2 Teacher

- **Purpose:** Records and maintains the academic information of the students assigned to them.
- **Main functions:** Views their assigned students, marks and edits attendance, enters and edits grades, and views the relevant academic records of those students.
- **Access level:** Restricted to the students assigned to that Teacher.
- **Information accessed:** Attendance, grades and academic records of assigned students only.
- **Technical proficiency (assumed):** Basic to moderate. The Teacher is expected to use standard web forms and tables for data entry and lookup.
- **Frequency of use (assumed):** Regular during the academic term, mainly for attendance and grade entry.
- **Important characteristics and limitations:** A Teacher cannot access students outside their assignments and cannot manage accounts or roles.

3.4.3 Student

- **Purpose:** Views their own academic information held by the institution.
- **Main functions:** Views their own profile, attendance, grades and academic records.
- **Access level:** Lowest privilege. Access is limited to the Student's own records and is read-only for academic information.
- **Information accessed:** Their own profile, attendance, grades and academic records only.
- **Technical proficiency (assumed):** Basic. The Student is expected to be able to log in and read information presented on screen.
- **Frequency of use (assumed):** Occasional, typically to check attendance or grades. Use may increase when results are published.
- **Important characteristics and limitations:** A Student cannot access any other student's records and cannot modify their own grades or attendance. Students are by far the most

numerous user class, and their access to the system depends on an Admin creating their account.

3.5 Operating Environment

TODO

3.6 Design and Implementation Constraints

TODO

3.7 Assumptions and Dependencies

TODO

4 External Interface Requirements

4.1 User Interfaces

The Student Record System is accessed by Admin, Teacher and Student users through a web-based frontend (Sections 2.3 and 5.6). Wireframes and mockups have not been produced for this version. Screen layouts and the interfaces for Sections 5.2 to 5.5 (records, attendance, grades, retrieval) are TBD until those sections are finalised.

Interfaces required by the current requirements

Interface	Used by	Logical characteristics	Reference
Login	All roles	Takes the registered email address and password. Authentication failures show one generic message that does not reveal which part was wrong. A user with a missing, expired or invalid session is directed here.	REQ-ONB-018, REQ-SEC-002, REQ-RBAC-006
Initial password setup	New Student/Teacher	Takes a password and a confirmation, using a single-use setup credential. On success the user is directed to log in separately and no session is created.	REQ-ONB-012 to 017

Interface	Used by	Logical characteristics	Reference
Account creation	Admin	Takes role, unique identifier (SRN or Staff ID), name, email and program/department details (fields in Appendix B). No password field is shown. A confirmation shows identifier, name and role only.	REQ-ONB-001 to 012, REQ-ONB-019
Account and role management	Admin	Provides update and deactivation of accounts and role assignment. Layout is TBD.	REQ-RBAC-007, 016 to 018
Assigned-student view	Teacher	Lists the students assigned to the Teacher. Detailed screens for attendance and grade entry are TBD (Sections 5.3 and 5.4).	REQ-RBAC-009
Own-record view	Student	Shows the Student's own profile, attendance, grades and academic records, read-only.	REQ-RBAC-012, 013

Role-specific behaviour

- The frontend displays only the functions permitted to the logged-in user's role (REQ-RBAC-004). The frontend is a convenience layer, and the backend enforces authorization on every request (REQ-RBAC-003).
- A request the user's role does not permit, including access to another user's record, produces an "Access Denied/Unauthorized" message and changes no data. Nonexistent and unauthorized records get the same response (REQ-RBAC-014).

Message conventions

- **Validation errors:** The message identifies every missing or invalid field and keeps the entered non-password values (REQ-ONB-008).
- **Duplicate identifier or email:** The message names the duplicated field and discloses no other data of the existing account (REQ-ONB-010).
- **Password rules and setup credential:** The message states which password rule failed, or that the credential is invalid (REQ-ONB-014, REQ-ONB-015).

- **Unexpected or authorization-check errors:** Only a generic message is shown, without stack traces, file paths, query text or other internal details (REQ-SEC-016, REQ-RBAC-015).
- **Passwords and hashes:** These are never displayed on any screen (REQ-SEC-004).

4.2 Software Interfaces

The implementation stack (languages, frameworks, database product, hosting operating system) has not yet been fixed in the SRS (Sections 2.4 and 2.5 and Appendix D are incomplete). Items below are therefore marked TBD where not established.

Component	Data exchanged	Purpose and conventions
Frontend Backend	Requests carrying user input, and responses carrying displayed records, confirmations and error messages. Sent from the frontend are the session identifier and, on state-changing requests, an anti-forgery token.	The frontend presents role-appropriate functions. The backend validates all input and authorizes every protected request through one common enforcement layer (REQ-SEC-011, REQ-SEC-013). The client cannot supply its own role, identity or ownership claim (REQ-RBAC-005). Frontend and backend frameworks: TBD.
Backend Database	Data categories established by the SRS: user accounts (identifier, name, email, role, status), password hashes, setup credentials, student records, attendance, grades, teacher–student assignments and audit-log entries.	Persistent storage for all system data. User input reaches the database only as bound parameters (REQ-SEC-014). Account creation is atomic (REQ-ONB-020), and uniqueness of SRN, Staff ID, Admin ID and email is guaranteed even for simultaneous requests (REQ-ONB-009). Database product, schema and the teacher–student assignment mechanism: TBD.
Backend Session store	Server-side session state and account status.	The server keeps session state and gives the client only an opaque identifier containing no role or personal data. Role and status are read from stored data on every protected request (REQ-SEC-008). The storage mechanism is TBD.

Component	Data exchanged	Purpose and conventions
Backend Password-hashing library	Plain password in, salted hash out.	Passwords are hashed with bcrypt, a unique salt and cost factor of at least 10 (REQ-SEC-003).
Backend Email delivery service	Outbound email carrying the initial-password setup credential to the user's registered address.	Used only for onboarding. No student records are sent by email (REQ-SEC-006, REQ-SEC-018). Email-sending mechanism or provider: TBD.
Continuous integration	Source code in, static-analysis findings out.	Static code analysis runs in CI, and a release is blocked while high-severity findings remain (REQ-SEC-019). Not a runtime interface.

No third-party APIs are used, and student and user data is not disclosed to third parties (REQ-SEC-018). The server operating system and other runtime dependencies are TBD pending Section 2.4.

4.3 Communications Interfaces

- **Client–server communication:** Users reach the system through a web browser. All client–server communication uses an encrypted connection (HTTPS). Unencrypted requests are rejected or redirected, and the session identifier is sent only over the encrypted connection and in a form not readable by client-side scripts (REQ-SEC-010). Protocol version and certificate arrangements are TBD.
- **Request/response model:** The frontend sends each user action to the backend and shows the backend's response. Every protected request carries the session identifier and is checked for authentication and authorization before processing (REQ-RBAC-003, REQ-RBAC-006). State-changing requests also carry a valid anti-forgery token tied to the session (REQ-SEC-015). A REST/JSON message format and API route conventions have not been established and are TBD.
- **Session communication:** The session identifier is opaque, is reissued at every successful login, and expires after 30 minutes of inactivity or 8 hours after login. A logged-out or expired session is rejected (REQ-SEC-008, REQ-SEC-009).
- **Backend–database communication:** The backend communicates with the database as described in Section 3.2. The connection mechanism and its security are TBD.
- **Email communication:** Email is used only to deliver the single-use initial-password setup credential, which is valid for 24 hours, to the user's registered address (REQ-SEC-006). Email contains no student records (REQ-SEC-018). Email verification, password-reset email and other notifications are out of scope in this version (Sections 5.1 and 6.3).
- **Other interfaces:** No communication with external systems or third-party services is required in this version. Transfer rates and synchronization requirements are not specified.

5 Analysis Models

TODO

6 System Features

6.1 Onboarding

6.1.1 Description and Priority

Description. Onboarding is how user accounts are created in the Student Record System. An authenticated Admin creates Student and Teacher accounts and assigns each account a role. Self-registration is out of scope. The Admin never sets or knows the user's password. The user sets their own password through a secure initial-password setup, then logs in separately using their registered email address and password.

Priority: High. Every other feature needs an existing, authenticated account.

Preconditions. - A pre-seeded Admin account exists (created during initial system setup). - The Admin performing onboarding is authenticated.

Inputs.

Account type	Unique identifier	Other required fields
Student	SRN	Name, email, academic/program details
Teacher	Staff ID	Name, email, department details
Admin (pre-seeded only)	Admin ID	Name, email, password

The Admin also selects the role (Student or Teacher). Field lengths, data types and the final field list are defined in Appendix B.

Assumptions and dependencies. - What each role may do after onboarding is defined in Section 5.6. - The hashing algorithm, session handling, setup-credential validity period and delivery mechanism are defined in Section 6.3 (TBD). - Email verification is out of scope. - Identifier and email comparisons ignore letter case and leading/trailing whitespace.

6.1.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Admin submits a valid account-creation request with a unique identifier and email.	Admin authenticated; required fields valid; identifier and email unique.	Account created as Active; password setup initiated; Admin sees a confirmation. No session is created for the new user.

#	Stimulus (trigger)	System validation	System response
SR-2	Admin submits a request with a missing or invalid field.	Field validation fails.	No account created; every failing field is identified; entered non-password values are retained.
SR-3	Admin submits an SRN, Staff ID, Admin ID or email already in use.	Uniqueness check finds a match.	Request rejected; existing account unchanged; error names the duplicated field.
SR-4	A non-Admin or unauthenticated user attempts account creation.	Authentication/role check fails.	Access denied; no account created.
SR-5	New user submits a valid password and matching confirmation using a valid setup credential.	Credential valid, unused, unexpired; password rules met.	Password hashed and stored; credential invalidated; user directed to log in separately.
SR-6	New user submits a password that breaks the rules, does not match, or uses an invalid/expired credential.	Password or credential check fails.	Nothing stored; message states what failed.
SR-7	User tries to log in before setting a password.	No password set for the account.	Login rejected with a generic authentication-failure message.

6.1.3 Functional Requirements

- **REQ-ONB-001:** The system shall allow only an authenticated Admin to create accounts, and shall reject any other attempt with an access-denied message and no account created.
- **REQ-ONB-002:** The system shall provide no self-registration function, and shall provide a pre-seeded Admin account created during initial system setup (handling of its initial credential is specified in Section 6.3).
- **REQ-ONB-003:** The system shall require the Admin to select exactly one role (Student or Teacher) for each new account, and shall store the selected role with the account.
- **REQ-ONB-004:** For a Student account, the system shall require SRN, name, email and academic/program details (as defined in Appendix B).
- **REQ-ONB-005:** For a Teacher account, the system shall require Staff ID, name, email and department details (as defined in Appendix B).
- **REQ-ONB-006:** The system shall neither require nor accept a password from the Admin when creating a Student or Teacher account.
- **REQ-ONB-007:** The system shall trim leading and trailing whitespace from text inputs,

and shall reject a request in which a required field is missing or blank, an email is not in a valid format (one "@" with non-empty parts and a "." in the domain), or any value violates the limits in Appendix B.

- **REQ-ONB-008:** When validation fails, the system shall create no account, shall display an error message identifying every invalid or missing field, and shall retain the entered non-password values.
- **REQ-ONB-009:** The system shall ensure that no two accounts share the same SRN, Staff ID, Admin ID or email address (compared without regard to letter case), including when two requests are submitted simultaneously.
- **REQ-ONB-010:** When a submitted identifier or email duplicates an existing account, the system shall reject the request, leave the existing account unchanged, and display an error message that names the duplicated field and discloses no other data of the existing account.
- **REQ-ONB-011:** The system shall set a successfully created account to Active with no approval step, and shall record the creation time and the creating Admin.
- **REQ-ONB-012:** Upon successful account creation, the system shall initiate a secure initial-password setup through which the user sets their own password, and shall never reveal that password to the Admin or any other user.
- **REQ-ONB-013:** The system shall issue the setup credential as single-use, bound to one account and valid for a limited period (period and delivery mechanism TBD in Section 6.3).
- **REQ-ONB-014:** The system shall reject a setup credential that is used, expired or invalid, shall set no password, and shall inform the user that the credential is not valid.
- **REQ-ONB-015:** The system shall require the password to be entered twice with matching entries, to have at least 8 characters, and to contain at least one uppercase letter, one lowercase letter, one digit and one special character; otherwise it shall not store the password and shall state which rule was not met.
- **REQ-ONB-016:** The system shall store a password only as a salted, adaptive, one-way hash (algorithm defined in Section 6.3), and shall not store, log or display it in plaintext.
- **REQ-ONB-017:** Upon successful password setup, the system shall invalidate the setup credential, shall create no authenticated session, and shall direct the user to log in separately.
- **REQ-ONB-018:** The system shall use the registered email address as the login identifier, and shall reject login for an account whose password is not yet set with the same generic message used for incorrect credentials.
- **REQ-ONB-019:** Upon successful account creation, the system shall show the Admin a confirmation containing the new account's identifier, name and role, and no password, hash or setup credential.
- **REQ-ONB-020:** The system shall create an account atomically, leaving no partial account if any step fails, and shall show a generic error message that exposes no internal details when an unexpected error occurs.

6.2 Edit Records

6.2.1 Description and Priority

TODO

6.2.2 Stimulus/Response Sequences

TODO

6.2.3 Functional Requirements

- REQ-REC-1:
- REQ-REC-2:

6.3 Mark Attendance (with constraints)

6.3.1 Description and Priority

TODO

6.3.2 Stimulus/Response Sequences

TODO

6.3.3 Functional Requirements

- REQ-ATT-1:
- REQ-ATT-2:

6.4 Mark Grades (with rubric)

6.4.1 Description and Priority

TODO

6.4.2 Stimulus/Response Sequences

TODO

6.4.3 Functional Requirements

- REQ-GRD-1:
- REQ-GRD-2:

6.5 Retrieval

6.5.1 Description and Priority

TODO

6.5.2 Stimulus/Response Sequences

TODO

6.5.3 Functional Requirements

- REQ-RET-1:
- REQ-RET-2:

6.6 Role-Based Access

6.6.1 Description and Priority

Description. Role-Based Access Control (RBAC) decides which functions and records each authenticated user may use. Every account has exactly one role: Admin, Teacher or Student. The role is assigned at account creation (Section 5.1), and only an Admin can change it. Permissions are enforced in both the frontend and the backend, and the backend is the authoritative enforcement layer. Anything not explicitly permitted to a role is denied. Authentication (Section 5.1 and Section 6.3) establishes who the user is, and authorization (this section) decides what that user may do.

Priority: High. Features 5.2 to 5.5 read and modify student data and depend on correct authorization.

Roles and permissions.

Role	Permitted	Scope boundary	Not permitted
Admin	Create, update, deactivate Student and Teacher accounts; assign and change roles; view and manage student records, attendance and grades; view all Teacher and Student records	All relevant records	None defined in this version
Teacher	View assigned students; mark and edit attendance and enter and edit grades for assigned students; view academic records of assigned students	Assigned students only	Students outside the assignment; account and role management
Student	View own profile, attendance, grades and academic records	Own records only	Other students' records; modifying own grades or attendance

Assumptions and dependencies. - Attendance rules (Section 5.3) and grading rules (Section 5.4) apply in addition to RBAC. - Session/token mechanism, audit-log retention and who may view the audit log are defined in Section 6.3 (TBD). - How a Teacher is assigned to a Student is TBD. RBAC only requires that the assignment exists. - TBD in this version: whether an Admin may deactivate their own or the last remaining Admin account, and how records and assignments are handled when an account's role changes.

6.6.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Admin requests an Admin-permitted function (e.g. view any record, edit grades, deactivate an account).	Session valid; account Active; role is Admin.	Request processed and result shown.
SR-2	Teacher requests an assigned student's record or attendance/grade function.	Session valid; role is Teacher; student is assigned to this Teacher.	Request processed, subject to Sections 5.3 and 5.4.
SR-3	Teacher requests a student who is not assigned to them.	Student not in this Teacher's assignment.	"Access Denied/Unauthorized"; no data returned; attempt logged.
SR-4	Student requests their own profile, attendance, grades or academic records.	Session valid; role is Student; record belongs to this Student.	Record displayed read-only.
SR-5	Student requests another student's record, or tries to modify their own grades or attendance.	Record not owned, or modification not permitted for Student.	"Access Denied/Unauthorized"; data unchanged; attempt logged.
SR-6	Any user requests a function not permitted for their role (e.g. Teacher changes a role).	Function not in the role's permission set.	"Access Denied/Unauthorized"; no change; attempt logged.
SR-7	Admin changes a user's role.	Requester is Admin; account exists; new role is valid.	Role updated and effective immediately; change recorded; confirmation shown.
SR-8	Deactivated user tries to log in or uses an existing session.	Account status is Deactivated.	Request rejected; no data returned; no new session created.
SR-9	Unauthenticated user, or user with an expired/invalid session, requests a protected function.	Authentication check fails.	Request rejected; user directed to log in; no data returned.

6.6.3 Functional Requirements

- **REQ-RBAC-001:** The system shall support exactly three roles (Admin, Teacher and Student) and shall associate every account with exactly one role at all times.

- **REQ-RBAC-002:** The system shall deny any function or record access that is not explicitly permitted to the requesting user's role.
- **REQ-RBAC-003:** The backend shall verify authentication and authorization (role and record scope) on every protected request before processing it, regardless of any frontend check.
- **REQ-RBAC-004:** The frontend shall display only the functions permitted to the logged-in user's role.
- **REQ-RBAC-005:** The system shall associate the authenticated user's role with the authenticated session, check it on every protected request, and not accept a client-supplied role, identity or ownership claim in its place.
- **REQ-RBAC-006:** The system shall deny a protected request whose session is missing, expired, invalid or inconsistent with the stored account, and for an unauthenticated user shall direct the user to log in without returning protected data.
- **REQ-RBAC-007:** The system shall allow an Admin to create, update and deactivate Student and Teacher accounts, and to assign and change roles.
- **REQ-RBAC-008:** The system shall allow an Admin to view and manage the records, attendance and grades of all students (subject to Sections 5.3 and 5.4) and to view all Teacher records.
- **REQ-RBAC-009:** The system shall allow a Teacher to view the list of students assigned to them, to mark and edit attendance and enter and edit grades for those students (subject to Sections 5.3 and 5.4), and to view those students' relevant academic records.
- **REQ-RBAC-010:** The system shall deny a Teacher any read or write access to a student who is not assigned to that Teacher, and shall evaluate the assignment on every request so a change takes effect on the next request (assignment mechanism TBD).
- **REQ-RBAC-011:** The system shall deny a Teacher the creation, update and deactivation of accounts and the assignment or change of roles.
- **REQ-RBAC-012:** The system shall allow a Student to view their own profile, attendance, grades and academic records.
- **REQ-RBAC-013:** The system shall deny a Student any access to another student's records, any modification of their own grades or attendance, and any Teacher or Admin function.
- **REQ-RBAC-014:** When a request is denied for lack of authorization, the system shall return a clear "Access Denied/Unauthorized" message, process no part of the request, leave data unchanged, and give the same response for a nonexistent record as for an unauthorized one.
- **REQ-RBAC-015:** When an authorization check cannot be completed because of an internal error, the system shall deny the request and show a generic error message that exposes no internal details.
- **REQ-RBAC-016:** The system shall allow only an Admin to assign or change a role, shall accept only Admin, Teacher or Student as the new role, and shall reject a role change for a nonexistent account with an error message.
- **REQ-RBAC-017:** The system shall apply a role change immediately, so that every later protected request (including from existing sessions) is authorized against the new role, shall keep exactly one role on the account, and shall record the affected account, previous role, new role, Admin and time.
- **REQ-RBAC-018:** The system shall allow only an Admin to deactivate an account, and shall invalidate all active sessions of that account immediately.
- **REQ-RBAC-019:** The system shall reject login by a deactivated account without creating a session, and shall deny every protected request made with a session of a deactivated account.

- **REQ-RBAC-020:** The system shall log every denied protected request and every failed authentication or session validation on a protected request (recording time, user identity and role if known, requested function or record, and outcome), shall not record passwords, hashes or session secrets in the log, and shall allow no role to modify or delete log entries through application functions.
- **REQ-RBAC-046:** The system shall not record passwords, password hashes or session secrets in the audit log.
- **REQ-RBAC-047:** Audit-log retention period and which roles may view the log are TBD and defined in Section 6.3.

7 Other Nonfunctional Requirements

7.1 Performance Requirements

TODO

7.2 Safety Requirements

TODO — likely N/A or brief for this project type, but don't just delete the section.

7.3 Security Requirements

TODO

7.4 Software Quality Attributes

7.5 Security Requirements

Security requirements for authentication, password protection, session handling, authorization enforcement, and protection of student data. They complete the items that Sections 5.1 and 5.6 defer to this section: hashing algorithm, session mechanism, setup-credential rules, and audit-log rules. Functional behaviour of onboarding and role-based access is in Sections 5.1 and 5.6 and is not repeated here. The numeric values below (for example timeouts and the lockout threshold) are team decisions for this project; the other numbers are reasonable defaults for a student record system.

Scope note. Password change and password reset are not part of this version. Email is used only to deliver the initial-password setup credential (Section 5.1).

Authentication and password protection - **REQ-SEC-001:** The system shall authenticate a user only by verifying the submitted password against the stored hash of the account matching the submitted registered email, and shall create a session only when verification succeeds for an Active account whose password has been set. - **REQ-SEC-002:** The system shall return the same generic authentication-failure message, with no indication of which part was wrong, for an unknown email, an incorrect password, a deactivated account, a locked account, and an account whose password is not yet set. - **REQ-SEC-003:** The system shall hash passwords with bcrypt using a unique random salt per password and a cost factor of at least 10, and shall verify passwords only by comparing against this hash. - **REQ-SEC-004:** The system shall never return, display, log or export a password or password hash in any response, screen, report or log entry. - **REQ-SEC-005:** The system shall lock login for an account for 15 minutes after 5 consecutive failed

login attempts, shall reset the failure count after a successful login, and shall show the generic message of REQ-SEC-002 while the lock is active. - **REQ-SEC-006:** The system shall generate each initial-password setup credential with a cryptographically secure random generator and at least 128 bits of randomness, deliver it to the user's registered email address, make it valid for 24 hours and for one use only, and invalidate it if the account is deactivated. - **REQ-SEC-007:** The system shall take the pre-seeded Admin's initial credential from the initial system setup, apply the password rules of REQ-ONB-015 and the hashing of REQ-SEC-003 to it, and shall not hard-code it in source code or commit it to the project repository.

Session and token security - **REQ-SEC-008:** The system shall keep session state on the server, shall give the client only an opaque session identifier generated with a cryptographically secure random generator and containing no role or personal data, shall issue a new identifier at every successful login, and shall obtain the user's role and account status from the server's stored data on every protected request. - **REQ-SEC-009:** The system shall expire a session after 30 minutes without activity or 8 hours after login, whichever comes first, and shall invalidate the session on the server immediately when the user logs out; an expired or logged-out session shall be rejected on every protected request. - **REQ-SEC-010:** The system shall carry all client-server communication over an encrypted connection, shall reject or redirect unencrypted requests, and shall send the session identifier only over that connection and in a form not readable by client-side scripts.

Authorization enforcement and record confidentiality - **REQ-SEC-011:** The system shall perform all authorization checks in the backend in one common enforcement layer through which every protected request passes, so that a protected function is unreachable without passing the checks of REQ-RBAC-003, and a newly added protected function is denied until a role permission is defined for it. - **REQ-SEC-012:** The system shall apply the access boundaries of Section 5.6 to every way of obtaining student data, including retrieval, search, lists, reports and exports, so that a result contains only records the requesting user is authorized to see.

Input validation and protection against common attacks - **REQ-SEC-013:** The system shall validate the type, length and format of every user-supplied input in the backend, regardless of any frontend validation, and shall reject invalid input without processing it. - **REQ-SEC-014:** The system shall never interpret user-supplied input as part of a query or command (it shall be passed to the data store only as bound parameters), and shall encode user-supplied data when displaying it so that it cannot run as script in the user's browser. - **REQ-SEC-015:** The system shall reject any state-changing request that does not carry a valid anti-forgery token tied to the user's session.

Secure error handling and audit logging - **REQ-SEC-016:** In every feature, the system shall show only generic error messages that contain no stack traces, file paths, query text, or other internal details, and shall record such details only in server-side logs. - **REQ-SEC-017:** In addition to the events required by REQ-RBAC-020, the system shall log successful and failed logins, account lockouts, account creations, role changes and account deactivations, each with time, acting user, affected account and outcome; the log shall be retained for at least 1 year and shall be viewable only by Admins, read-only.

Data protection and security validation - **REQ-SEC-018:** The system shall use student and user data only within the Student Record System, shall not disclose it to third parties, and shall not include student records in any email. - **REQ-SEC-019:** The project's continuous integration shall run static code analysis, and a release shall not be made while high-severity security findings

remain unresolved. TODO

7.6 Business Rules

TODO

8 Other Requirements

TODO

9 Appendix A: Glossary

Term	Definition
Student	
Section	
Term	
Rubric	

10 Appendix B: Field Layouts

10.1 Student

Field	Length	Data Type	Description	Is Mandatory

11 Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID

12 Appendix D: Stakeholder Discussion Log

This project had no real customer, so a hypothetical stakeholder was used to ground scope and design decisions in the same back-and-forth a real software engineering team would have. The discussions below were conducted over roughly four weeks, and the decisions recorded here are referenced from Sections 1.3, 2.4, 2.5 and 2.6 of this SRS.

12.1 Persona

Name / Role: Dr. Kobe Bryant, Vice Chancellor

Context: Dr. Kulkarni oversees a university with approximately 10,000 students and staff combined. Student records, attendance and grades are currently maintained across spreadsheets and departmental registers, which makes consolidated reporting slow and access control inconsistent. He has commissioned a single Student Record System for the whole institution.

12.2 Discussion 1: Discovery Call (Monday, 14 September 2026)

Topics covered: current pain points, user roles, must-have features.

Q: What problems with the current record-keeping process are driving this request?

A: Records are scattered across departments. Producing a consolidated report for a single student means collecting data from several offices, and there is no reliable way to tell who changed a record or when. Attendance is often entered days late.

Q: Who will use the system, and what should each group be able to do?

A: Three groups: administrative staff who register students and maintain records, faculty who mark attendance and grades for their own courses, and students who should be able to view only their own records. Faculty must not be able to see or edit other faculty's courses.

Q: Are there rules on attendance entry that the system should enforce?

A: Yes. Attendance should only be markable for a class session that has actually been scheduled, a student cannot be marked present in two overlapping sessions, and faculty should be able to correct an entry only within a limited window after the session. After that, only an administrator can amend it, with the change logged.

Q: How are grades assigned today?

A: Each course is evaluated against a rubric of weighted components such as assignments, internal tests and a final exam. Rubrics differ between courses, so the system cannot assume a single fixed scheme.

Decisions made:

- Three user roles: Administrator, Faculty and Student, with role-based access enforced by the system.
- Core features confirmed: onboarding, editing records, attendance with constraints, rubric-based grading, retrieval, and role-based access.
- Attendance edit window and overlap rules will be specified as concrete requirements (see Section 5.3).
- The grading rubric will be configurable per course by faculty rather than hardcoded (see Section 5.4).
- Every change to a record, attendance entry or grade must be traceable to a user and a timestamp.

12.3 Discussion 2: Deployment and Constraints (Thursday, 24 September 2026)

Topics covered: cloud versus self-hosted deployment, cost, expected load, data sensitivity, technology stack.

Q: Should the system be hosted on a cloud provider such as AWS, GCP, Azure or Firebase, or run on the institution's own infrastructure?

A: We initially leaned towards the cloud because it avoids buying hardware and gives a public URL immediately. The team walked through the trade-offs, and several points changed our view.

First, cost. A cloud deployment carries a recurring bill for compute, managed database, storage and backups that grows with usage and never ends. The institution already owns server capacity and a data centre room with spare capacity, so self-hosting is a one-time setup effort with no ongoing subscription.

Second, maintenance. The institution's existing IT team already runs servers and databases for other systems, so operating one more containerised application is well within their abilities and does not require hiring or training.

Third, the main advantage of the cloud does not apply here. Cloud platforms are most valuable for load balancing and low latency across large geographic areas, but nearly all users of this system are students and staff who regularly visit the campus and can reach a server on the campus network. There is no geographically distributed user base to serve.

Q: If cloud had been chosen, would a conventional backend or a serverless framework have been preferred?

A: Serverless was discussed and set aside even before the cost argument. The system depends on session-based authentication, role checks on every request and validation of attendance rules, all of which are simpler to build, test and reason about in a conventional long-running backend. Cold starts would also work against the response-time target below.

Q: What scale should the system be designed for?

A: About 10,000 users in total. Most of the time only a small fraction will be active, but there are predictable peaks: faculty marking attendance at the start and end of class periods, and students checking results when grades are published. The system should handle a few hundred concurrent users comfortably during these peaks.

Q: What are the expectations on response time?

A: Routine actions such as marking attendance or looking up a student should feel instant. A target of under two seconds for record retrieval under normal load was agreed.

Q: Are there data privacy or retention constraints?

A: Student records are personal data and must be protected accordingly, including compliance with India's Digital Personal Data Protection Act, 2023. Access must be limited by role, passwords must never be stored in plain text, and the database must be backed up regularly. Records are to be retained for the duration of a student's enrolment and for a period afterwards, as set by the university's records policy.

Q: Which technology stack should be used?

A: The institution has no preference, provided it is open source and maintainable by the existing IT team. The team proposed React for the frontend, Flask for the backend and a relational database, since student, course, attendance and grade data is inherently relational. MySQL will be used for deployment, with SQLite acceptable for local development and testing.

Decisions made:

- Deployment model: self-hosted on the institution's own infrastructure, packaged with Docker so the IT team can deploy and update it consistently. Cloud and serverless options were considered and rejected for cost, operational simplicity and lack of a geographically distributed user base.
- Technology stack: React (frontend), Flask (backend), MySQL (production database), SQLite (development and testing).
- Design capacity: approximately 10,000 total users, with a few hundred concurrent users at peak.
- Performance target: record retrieval in under 2 seconds under normal load.
- Security: hashed passwords, role-based access enforced on the server, audit trail of changes, regular database backups.

12.4 Discussion 3: Sign-off (Tuesday, 6 October 2026)

Topics covered: final scope confirmation, items excluded from this release, acceptance.

Q: Having reviewed the scope, is there anything that must be added before work begins?

A: No additions. The six features agreed in Discussion 1 cover the immediate need.

Q: Are there features that can safely be deferred?

A: Yes. Fee management, timetable generation, a parent portal and integration with the university's single sign-on service are out of scope for this release. They can be considered in a later phase, and the design should not make them hard to add.

Q: Is the proposed deployment plan acceptable to the institution's IT team?

A: Yes. The IT team has reviewed the self-hosted, containerised approach and confirmed they can host and maintain it on existing infrastructure.

Decisions made:

- Scope for this release is the six confirmed features: onboarding, editing records, attendance with constraints, rubric-based grading, retrieval, and role-based access.
- Out of scope: fee management, timetable generation, parent portal, single sign-on integration.
- The extensibility of the design to accommodate deferred features later is a stated design goal.
- The stakeholder approved the SRS scope, deployment model and technology stack as recorded in this appendix.